INSLIB — Getting Started in C

A beginner's guide to the error-state Kalman filter

Jan Zwiener

2026

Contents

1 INSLIB in C **1**
- 1.1 What you get . . . 1
- 1.2 Design . . . 2
- 1.3 First program . . . 2
- 1.4 The three calls you actually use . . . 5
- 1.5 Starting without GNSS: lighthouse / UWB / mocap . . . 6
- 1.6 Adding more sensors . . . 6
- 1.7 Calibrating your IMU (no fixture needed) . . . 7
- 1.8 Beyond the bare filter: `nav_suite` . . . 7
- 1.9 Where to look next . . . 7

1 INSLIB in C

INSLIB is a small, portable navigation-filter library written in C11. It fuses an IMU (accelerometer + gyroscope) with other aiding sensors - GNSS, a magnetometer, a barometer, or an indoor position system such as Lighthouse / UWB / motion capture - into one estimate of **where you are** and **which way you are pointing**.

This tutorial gets you from nothing to a running solution in one file of plain C. No prior Kalman-filter knowledge is assumed.

1.1 What you get

The core filter (`INS`) is a **15-state error-state Kalman filter (ESKF)**. "15 states" means it continuously estimates:

States	Quantity	Frame
0–2	position	local NED (north-east-down)
3–5	velocity	NED
6–8	attitude (roll/pitch/yaw error)	body → NED
9–11	accelerometer bias	body
12–14	gyroscope bias	body

No need to touch the internals. You feed measurements in and read a position/velocity/attitude solution out.

1.2 Design

- **No heap, no OS.** The whole filter is one caller-owned `struct` (`ins_t`). You zero it, initialise it, and feed it. It never calls `malloc`. That is basically a drop-in library for microcontrollers.
- **Conventions.** Body frame is **FRD** (x forward, y right, z down), navigation frame is **NED**. Quaternions are Hamilton with `q[0] = w`. Timestamps are `int64` microseconds. Angles are radians.
- **You only set what you care about.** Almost every configuration field treats `0` as "pick a sensible default", so a zeroed options struct already gives you a working (but not optimal) filter.

1.3 First program

Here is a complete program. It creates a filter, feeds it ten seconds of a (stationary, level) IMU at 100 Hz with a once-per-second GNSS fix, and prints the resulting position and attitude.

```c
/* hello_ins.c - minimal INSLIB example: fuse a stationary IMU with a
 * GNSS position fix and print the navigation solution. */
#include <math.h>
#include <stdio.h>

#include "ins.h"
#include "geodetic_toolbox.h"

int main(void)
{
    /* 1. The filter lives in one caller-owned struct. Zero it first. */
    ins_t ins = {0};

    /* Example data: a spot in Darmstadt, 190 m ellipsoidal. The
     * filter works in a local NED frame, this is where its origin sits. */
    const double lat = DEG2RAD(49.867378), lon = DEG2RAD(8.638174), h = 190.0;

    /* 2. Initial values and noise assumptions. Zeros pick sane consumer
     *    MEMS defaults, so we only set what we care about. The position fixes the
     *    local-frame origin (a coarse guess is fine - auto_init refines the
     *    solution from the first real fix).
     *    Initial position is has 5 m precision, initial velocity is zero
     *    with a precision of 1 m/s, initial roll/pitch/yaw is 5° deg precise.
     */
    ins_init_t init = {0};
    init.llh[0] = lat;
    init.llh[1] = lon;
    init.llh[2] = h;
    init.pos_init_stddev_m = 5.0f; /* [m]   trust the first fix ~5 m  */
    init.vel_init_stddev_mps = 1.0f; /* [m/s] trust in init. velocity */
```

```
    init.rpy_init_stddev_rad[0] = DEG2RAD(5.0f);
    init.rpy_init_stddev_rad[1] = DEG2RAD(5.0f);
    init.rpy_init_stddev_rad[2] = DEG2RAD(5.0f);

    /* 3. Options. auto_init lets the filter bootstrap its start time, level
     *    (roll/pitch) and position from the first coherent IMU+GNSS it sees.
     *    Everything left at 0 falls back to best-effort reasonable defaults.
     *    But check the details, often the default is not good enough.
     */
    ins_options_t opt = {0};
    opt.auto_init = true;

    if (ins_init(&ins, &init, &opt) != 0) {
        fprintf(stderr, "ins_init failed\n");
        return 1;
    }

    /* 4. Feed measurements. 100 Hz IMU for 10 s, a 1 Hz GNSS fix on top.
     *    At rest and level the accelerometer measures specific force
     *    f = -g: in body FRD (z down) that is +g pointing "up" == -z. */
    const float dt = 0.01f; /* 100 Hz */
    for (int k = 1; k <= 1000; ++k) {
        ins_time_us_t t_us = (ins_time_us_t)(1.0 * 1e6 * k * dt);

        ins_measurements_t m = {0};
        m.timestamp        = t_us;
        m.strapdown_dt_sec = dt;

        m.acc.is_valid = true;
        m.acc.data[0]  = 0.0f;
        m.acc.data[1]  = 0.0f;
        m.acc.data[2]  = -9.81f; /* specific force at rest, FRD */

        m.gyr.is_valid = true;
        m.gyr.data[0]  = 0.0f;
        m.gyr.data[1]  = 0.0f;
        m.gyr.data[2]  = 0.0f;

        /* GNSS once per second: position + a diagonal NED covariance
         * (here 1.5 m horizontal, 2.5 m vertical 1-sigma -> variance in m^2,
         * inside the 3D entry gate, see "Warm-up" below).
         * The fix goes in as the receiver reports it, latitude, longitude
         * and height, which is the form the filter fuses in. A source that
         * is natively ECEF converts once with ins_ecef_to_latlonh(). */
        if (k % 100 == 0) {
            m.gnss_pos.is_valid   = true;
            m.gnss_pos.llh[0]     = lat;
```

```
            m.gnss_pos.llh[1]     = lon;
            m.gnss_pos.llh[2]     = h;
            m.gnss_pos.Qll_ned[0] = 1.5f * 1.5f; /* var N */
            m.gnss_pos.Qll_ned[4] = 1.5f * 1.5f; /* var E */
            m.gnss_pos.Qll_ned[8] = 2.5f * 2.5f; /* var D */
        }

        ins_update(&ins, &m);
    }

    /* 5. Read the solution. */
    if (!ins_is_ready(&ins)) {
        printf("filter still warming up\n");
        return 0;
    }

    double llh[3];
    float  roll, pitch, yaw;
    ins_get_latlonh(&ins, llh); /* lat [rad], lon [rad], h over the ellipsoid */
    ins_get_rpy(&ins, &roll, &pitch, &yaw);

    printf("position : lat %.6f deg  lon %.6f deg  h %.1f m\n",
           RAD2DEG(llh[0]), RAD2DEG(llh[1]), llh[2]);
    printf("attitude : roll %.2f  pitch %.2f  yaw %.2f deg\n",
           RAD2DEG(roll), RAD2DEG(pitch), RAD2DEG(yaw));
    return 0;
}
```

1.3.1 Build and run

INSLIB has no build-system dependency, just compile its `.c` files alongside your project. From the repository root:

```
cc -std=c11 -D_GNU_SOURCE \
   -Isrc -IKFCore/c -IKFCore/c/navigation_tools \
   hello_ins.c \
   src/ins.c src/log.c src/geodetic_toolbox.c src/magnetic_model.c \
   KFCore/c/linalg.c KFCore/c/kalman_udu.c KFCore/c/miniblas.c \
   -lm -o hello_ins
./hello_ins
```

Expected output:

```
position : lat 49.867378 deg  lon 8.638174 deg  h 190.0 m
attitude : roll 0.00  pitch 0.00  yaw 0.00 deg
```

The position matches the fix and the attitude comes out level, the filter levelled itself from the accelerometer during warm-up.[1] The log also prints a few warnings (`initial heading unknown`,

[1] A real gyro never measures exactly zero, the Earth's rotation alone is 15 deg/h.

`gyro sample identical ...`, `yaw not aided ...`). They are expected here: the example has no magnetometer or GNSS velocity, so the heading is unobservable, and the synthetic IMU is noise-free. The `yaw` value is not meaningful.

> `-D_GNU_SOURCE` is only needed so the standard headers expose `M_PI` to the library sources, it is not an INSLIB requirement of your own code.

1.4 The three calls you actually use

Everything in the example reduces to a tiny loop:

1. **`ins_init(&ins, &init, &opt)`** - once, after zeroing the struct.
2. **`ins_update(&ins, &m)`** - once per epoch, with a filled `ins_measurements_t`.
3. **`ins_get_* / ins_is_ready`** - read the answer whenever you want it.

`ins_measurements_t` is a bundle: set the whole struct to `{0}` and fill in only the fields you have this epoch. Each sub-measurement has an `is_valid` flag, the filter consumes exactly the ones you flag. Missing a sensor for an epoch is normal - just leave it invalid.

1.4.1 Warm-up

`ins_is_ready()` returns `false` until the filter has levelled and had a run of good fixes (a few seconds). Read the solution only once it is `true`. With `auto_init` enabled the filter bootstraps itself from the first coherent IMU+fix pair, you do not have to hand it an accurate initial attitude or position. Only the heading can be tricky, if there is no magnetometer, a GNSS+IMU system with a wrong heading will produce nonsense data until enough movement allows the system to converge.

The fixes have to be good enough for a 3D solution, which is stricter than being good enough to fuse: the entry gate (`opt.gnss_start_max_*`, by default 2 m horizontal / 3 m vertical position and 0.25 m/s velocity 1-sigma) must hold for `opt.gnss_init_dwell_sec`. If your receiver reports a coarser accuracy than that, raise the entry gate - otherwise the filter keeps consuming fixes without ever starting (it says so in the log, once per 30 s).

The same idea runs in the other direction: if every fix over `opt.gnss_stop_dwell_sec` (default 10 s) is worse than the exit gate (`opt.gnss_stop_max_*`, default 5 m horizontal / 7 m vertical position and 0.4 m/s horizontal / 0.5 m/s vertical velocity), the filter gives up the 3D solution - it stops rather than coasting on: it re-arms into the collecting state, so `ins_get_position_local()` and friends report nothing at all instead of a position that keeps drifting off bad fixes. Coming back is a full re-bootstrap through the entry gate, with the IMU biases carried over at an inflated uncertainty and roll/pitch/yaw re-seeded from the parallel AHRS if you run `nav_suite`.

One more thing your GNSS receiver does not tell you: how much its reported position and velocity have in common. They come out of one internally coupled solution, but the correlation between them is not in any common output message, so fusing both blocks every epoch counts the same information twice and leaves the filter more confident than the data warrants. `opt.gnss_pos_decimation` spends only one of them per epoch: every Nth epoch that offers both fuses the position, the rest fuse the velocity alone. Set it to 1 if you want a combined fuse.

1.5 Starting without GNSS: lighthouse / UWB / mocap

You do not need GNSS. Any system that reports a local NED position - SteamVR lighthouse, UWB, a motion-capture rig, even a total station can aid the filter through `local_pos` instead. You still give a *coarse* Earth anchor at init (`llh`), but only so the filter knows the local gravity and where true north is, it is never used as an aiding measurement.

The only change from the example above is what you put in the measurement each epoch:

```
/* A 10 Hz indoor tracker reporting position in the filter's local NED
 * frame (origin = the position you gave at init). 1 cm 1-sigma per axis. */
if (k % 10 == 0) {
    m.local_pos.is_valid   = true;
    m.local_pos.pos_ned[0] = 1.5f;  /* north [m] */
    m.local_pos.pos_ned[1] = 0.0f;  /* east  [m] */
    m.local_pos.pos_ned[2] = 0.0f;  /* down  [m] */
    m.local_pos.Qll_ned[0] = 0.01f * 0.01f;
    m.local_pos.Qll_ned[4] = 0.01f * 0.01f;
    m.local_pos.Qll_ned[8] = 0.01f * 0.01f;
}
```

Read the local position back with `ins_get_position_local(&ins, pos_ned)`. Aligning the tracker's own coordinate frame with the filter's NED frame (origin offset + rotation) is your responsibility - INSLIB assumes the `pos_ned` you hand it is already in its frame.

1.6 Adding more sensors

The same `ins_measurements_t` carries everything, flip on the fields you have:

- `m.gnss_vel` - a GNSS velocity in NED (with its own covariance). Velocity aiding is what makes the heading observable while moving.
- `m.mag` - a magnetometer sample. Call `ins_set_magnetic_model_from_position(&ins, lat, lon, year)` once (e.g. on your first fix) so yaw is referenced to **true** north via the World Magnetic Model. Leave `m.mag.Qll_diag` at zero unless you have characterised the sensor *on your platform*: the filter then uses its own noise default, which is deliberately weak (roughly the horizontal field itself, so about a radian of heading per sample), and throttles the fusion to 1 Hz (`opt.magnetometer_min_delay_ms`). The magnetometer is meant to keep yaw from drifting away over minutes, not to steer it every epoch - the gyro is better at that, and the field near motors and steel is not as good as its noise figure suggests. A hard-iron offset is a different problem: no variance fixes it, only `opt.mag_fixed_bias` / `opt.estimate_mag_bias` do.
- `m.yaw` - an absolute heading in **radians** (dual-antenna GNSS, mocap pose, gyro compass). Either convention works, `[-pi, pi]` or `[0, 2*pi)`, the filter normalises it. A value beyond one full turn is taken as a unit or unwrapping mistake (degrees in a radian field, an accumulated course), dropped, and counted in `n_invalid_input`. If the heading aiding does nothing, check that counter first. A dual-antenna receiver measures the azimuth of its antenna baseline, which is the yaw only for antennas lined up along the body x axis. For any other mounting convert it first with `ins_yaw_from_baseline_heading(heading, roll, pitch, baseline_b, &yaw)` (`geodetic_toolbox.h`), using the attitude you currently have.
- `m.zero_velocity_update` / `m.zero_rotation_update` - tell the filter you know you are

standing still, it re-estimates its IMU biases. (An automatic detector does this by default.)

- **`m.gnss_delay_ms`, `m.local_pos_delay_ms`, `m.yaw_delay_ms`** - if a fix is a known number of milliseconds old (latency) the filter anchors the correction at the right point in its state history. This is essential for agile vehicles. `m.gnss_delay_us`, `m.local_pos_delay_us`, `m.yaw_delay_us` and `m.speed_delay_us` add a microsecond part on top: if you attach a fix to the next IMU sample after it arrived, pass how much older it is than that sample, otherwise the solution trails the truth by speed times that age.

Every field is documented in `src/ins.h` - this header is the reference for the whole API.

1.7 Calibrating your IMU (no fixture needed)

A cheap MEMS IMU straight out of the box has bias, scale factor and axis misalignment errors that the filter can only partly estimate on its own, and a magnetometer is off by hard and soft iron. INSLIB includes a calibration tool that measures all of it without a turntable or a calibration rig, from one recording of your own sensor:

1. Leave the unit completely still for about 20 s.
2. Then pick it up, turn it to a new attitude (any, it does not have to be level), put it down and hold it for a few seconds. Repeat 20 to 30 times.
3. Log this as `imu.csv` (`t_us, gyr xyz [rad/s], acc xyz [m/s^2]`) and, if you have one, `mag.csv` (`t_us, mag xyz [uT]`), FRD axes, one clock.

```
python3 tools/inslib_imu_calib.py --csv mysession/ -o config.yaml
```

The result lands in `config.yaml` as column-major 3x3 matrices and bias vectors in exactly the model the filter applies, `corrected = M * (raw - fixed_bias)`. Copy them into `opt.imu_acc_misalignment`, `opt.imu_gyr_misalignment`, `opt.imu_acc_fixed_bias`, `opt.imu_gyr_fixed_bias` and, for the magnetometer, `opt.mag_misalignment` / `opt.mag_fixed_bias`. The CSV path works with any IMU.

1.8 Beyond the bare filter: `nav_suite`

`ins` alone drops its position solution during a long GNSS outage. The `nav_suite` wrapper (see `src/nav_suite.h`) runs `ins` **plus** two attitude-only filters (ARS/AHRS) and a barometric vertical channel in parallel, and arbitrates the best available answer: `FULL` -> `COASTING` -> `ATTITUDE_ONLY` -> `NONE`. If you want graceful degradation out of the box, start there, the measurement struct and conventions are identical. With a magnetometer, call `nav_suite_set_magnetic_model_position(&suite, lat, lon, year)` instead of the `ins` function: it reaches the AHRS as well, at any time, right after `nav_suite_init()` included.

1.9 Where to look next

- `src/ins.h` - every option, measurement field and accessor, documented.
- `src/nav_suite.h` - the "best available solution" wrapper.
- `tests/test_ins_core.c` - dozens of small, readable usage scenarios.
- `python_tutorial.md` - the same library from Python in a few lines.